Ch 08: String Methods, testing, searching and manipulating strings

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Testing, Searching and Manipulating Strings

Strings

Python provides operators and methods for testing strings, searching the contents of strings, and getting modified copies of strings.

- You can use the `in` operator to determine whether one string is contained in another string
  - General format: `string1 in string2`
    - `string1` and `string2` can be string literals or variables referencing strings
- Similarly you can use the `not in` operator to determine whether one string is not contained in another string

EAST TEXAS A&M

String Testing Methods

- Strings in Python have many types of methods, divided into different types of operations
  - General format: `mystring.method(arguments)`
- Some methods test a string for specific characteristics
  - Generally Boolean methods, that return `True` if a condition exists, and `False` otherwise
- There are many more string methods, see Python documentation (Python, 2026)

Table 1: Some string testing methods

Method	Description
`isalnum()`	Returns true if the string contains only alphabetic letters or digits.
`isalpha()`	Returns true if the string contains only alphabetic letters .
`isdigit()`	Returns true if the string contains only numeric digits.
`islower()`	Returns true if all of the alphabetic letters in the string are lowercase.
`isspace()`	Returns true if the string contains only whitespace characters. Whitespace characters are spaces, newlines and tabs.
`isupper()`	Returns true if all of the alphabetic letters in the string are uppercase.

EAST TEXAS A&M

String Modification Methods

- Some methods modify string, returning new string after modifications

Table 2: Some string modification methods

Method	Description
`lower()`	Returns a copy of the string with all alphabetic letters converted to lowercase.
`lstrip()`	Returns a copy of the string with all leading whitespace characters removed.
`lstrip(char)`	The `char` argument is a string containing characters. Returns a copy of the string with all instances of `char` that appear at the beginning of the string removed.
`rstrip()`	Returns a copy of the string with all trailing whitespace characters removed.
`rstrip(char)`	The `char` argument is a string containing characters. Returns a copy of the string with all instances of `char` that appear at the end of the string removed.
`strip()`	Returns a copy of the string with all leading and trailing whitespace characters removed.
`strip(char)`	The `char` argument is a string containing characters. Returns a copy of the string with all instances of `char` that appear at the beginning and end of the string removed.
`upper()`	Returns a copy of the string with all alphabetic letters converted to uppercase.



EAST TEXAS A&M

String Search and Replace Methods

- Programs commonly need to search for substrings
  - `startswith(substring)`: checks if the string starts with given substring
  - `endswith(substring)`: checks if the string ends with given substring
  - `find(substring)`: searches for `substring`, returns lowest index of the substring or -1
  - `replace(substring, new_string)`: returns a copy of string where every occurrence is replaced

Table 3: Some string search and replace methods

Method	Description
`endswith(substring)`	The method returns true if the string ends with `substring`.
`find(substring)`	The method returns the lowest index in the string where `substring` is found, or -1 if not found.
`replace(old, new)`	The method returns a copy of the string with all instances of `old` replaced by `new`.
`startswith(substring)`	The method returns true if the string starts with with `substring`.



EAST TEXAS A&M

The Repetition Operator

- **Repetition operator**: makes multiple copies of a string and concatenates them together
  - The * symbol is a repetition operator when applied to a string (lhs) and an integer (rhs)
  - General format: `string_to_copy * n`
  - Variable references a new string which contains multiple copies of the original string

```
my_string = 'Hello ' * 5
print('-' * 30)
print(my_string)
print('-' * 30)
```

```
------------------------------
Hello Hello Hello Hello Hello
------------------------------
```

EAST TEXAS A&M

Splitting a String

- `split()` method: returns a list containing the words in the string
  - By default, uses space as separator
  - Can specify a different separator by passing it as an argument to the `split()` method

```
my_string = 'One two three four'
word_list = my_string.split()
print(word_list)
['One', 'two', 'three', 'four']

my_string = '1/2/3/4/5'
number_list = my_string.split('/')
print(number_list)
['1', '2', '3', '4', '5']
```



String Tokens

Sometimes we tokenize a string by defining substrings separated by a (set of) special characters

- Example: `peach raspberry strawberry vanilla`
- This string contains the substrings `['peach', 'raspberry', 'strawberry', 'vanilla']`
- The substrings are separated by a space delimiter character
- The substrings are known as **tokens** and the separating character is known as the **delimiter**

- **Tokenizing** is the process of breaking a string into tokens
  - When you tokenize a string, you extract the tokens and store them as individual items
  - In Python you can use the split method to tokenize a string

```
str = 'peach raspberry strawberry vanilla'
tokens = str.split()
print(tokens)
['peach', 'raspberry', 'strawberry', 'vanilla']

my_address = 'www.example.com'
tokens = my_address.split('.')
print(tokens)
['www', 'example', 'com']
```

EAST TEXAS A&M

Summary

- You can use `in` and `not in` operator, as with other sequences, to test if substring contained in another string
- String testing methods like `isdigit()` and `islower()` return true if all characters of string are of the type being tested
- String modification methods like `strip()` are useful for removing beginning and trailing whitespace and other conversions
- String search and replace methods can be used to find where in a string a substring is found
- You can tokenize strings in Python using the `split()` method, which will return a list of tokens split by the delimiter character(s).



References

Python. (2026). *Python 3 standard library documentation*. Retrieved August 13, 2026, from https://docs.python.org/3/

EAST TEXAS A&M